Full Stack Development with MERN

Project Documentation

---

1. Introduction

- **Project Title: Streamlining Ticket Assignment For Efficient Support Operations**
- **Team Members:**
  - **RAMAKRISHNAN M – Full Stack Developer / Project Lead**
  - **Team Member 2 – Backend Developer**
  - **Team Member 3 – Frontend Developer**

---

2. Project Overview

Purpose

The project aims to develop an automated ticket assignment system that improves support operations by reducing manual effort, minimizing errors, and ensuring faster issue resolution.

Features

- **User Registration & Login**
- **Ticket Creation & Tracking**
- **Automated Ticket Assignment**
- **Role-Based Access Control (RBAC)**
- **Dashboard for Monitoring**
- **Notifications & Status Updates**

---

3. Architecture

Frontend (React)

- Built using React.js with component-based architecture
- Pages include Login, Register, Dashboard, Ticket Form
- Uses Axios for API communication
- State managed using React Hooks

Backend (Node.js & Express.js)

- RESTful APIs built using Express.js
- Handles:
  - Authentication
  - Ticket management
  - Assignment logic
- Middleware used for validation and security

Database (MongoDB)

Collections:

User Collection

```
{
 "name": "string",
 "email": "string",
```

"password": "string",

"role": "string"

}

Ticket Collection

{

"title": "string",

"description": "string",

"category": "string",

"status": "Open/In Progress/Closed",

"assignedTo": "userId"

}

---

4. Setup Instructions

Prerequisites

- Node.js
- MongoDB
- Git

Installation Steps

```
# Clone repository
git clone
   https://github.com/your-repo/project.git

# Navigate to project
cd project

# Install frontend dependencies
cd client
npm install

# Install backend dependencies
cd ../server
npm install
```

---

5. Folder Structure

Client (React Frontend)

```
client/
├── src/
│   ├── components/
│   ├── pages/
```

```
│   ├── services/
│   └── App.js
└── package.json
```

Server (Node Backend)

```
server/
├── models/
├── routes/
├── controllers/
├── middleware/
└── server.js
```

---

6. Running the Application

Frontend

```
cd client
npm start
```

Backend

```
cd server
npm start
```

---

7. API Documentation

Register User

- **POST `/api/register`**

Request Body:

```
{
 "email": "user@example.com",
 "password": "123456"
}
```

Login User

- **POST `/api/login`**

Create Ticket

- **POST `/api/tickets`**

Get Tickets

- **GET `/api/tickets`**

Update Ticket

- **PUT `/api/tickets/:id`**

---

8. Authentication

- **Uses JWT (JSON Web Token) authentication**
- **Token generated during login**
- **Stored in browser local storage**
- **Middleware verifies token for protected routes**
- **Role-based authorization ensures proper access**

---

9. User Interface

Includes:

- **Login Page**
- **Registration Page**
- **Dashboard**
- **Ticket Creation Form**
- **Ticket Status Tracking**

(Attach screenshots or GIFs here in your report)

---

10. Testing

- **Manual testing for UI and functionality**
- **API testing using Postman**
- **Tested modules:**
  - **Authentication**
  - **Ticket creation & assignment**
  - **Role-based access**

---

11. Screenshots / Demo

- **Add screenshots of:**
  - **Login Page**
  - **Dashboard**
  - **Ticket Creation**
  - **Ticket Assignment**
- **Demo Link: (Add your project demo link)**

---

12. Known Issues

- **Limited automation rules**
- **No AI-based classification**
- **UI can be further enhanced**

---

13. Future Enhancements

- **AI-based ticket classification**
- **Chatbot integration**
- **Advanced analytics dashboard**
- **Mobile application support**

Conclusion

This MERN stack project successfully implements a full-stack solution for automating ticket assignment, improving efficiency, and enhancing the overall support system experience.

Project Documentation

---

1. Introduction

- **Project Title: Streamlining Ticket Assignment For Efficient Support Operations**
- **Team Members:**
  - **Dhanasekar P – Full Stack Developer / Project Lead**
  - **Team Member 2 – Backend Developer**
  - **Team Member 3 – Frontend Developer**

---

2. Project Overview

Purpose

The project aims to develop an automated ticket assignment system that improves support operations by reducing manual effort, minimizing errors, and ensuring faster issue resolution.

Features

- **User Registration & Login**
- **Ticket Creation & Tracking**
- **Automated Ticket Assignment**
- **Role-Based Access Control (RBAC)**
- **Dashboard for Monitoring**
- **Notifications & Status Updates**

---

3. Architecture

Frontend (React)

- **Built using React.js with component-based**

architecture
- Pages include Login, Register, Dashboard, Ticket Form
- Uses Axios for API communication
- State managed using React Hooks

Backend (Node.js & Express.js)

- RESTful APIs built using Express.js
- Handles:
  - Authentication
  - Ticket management
  - Assignment logic
- Middleware used for validation and security

Database (MongoDB)

Collections:

User Collection

```
{
 "name": "string",
 "email": "string",
 "password": "string",
 "role": "string"
}
```

Ticket Collection

```
{
 "title": "string",
 "description": "string",
 "category": "string",
 "status": "Open/In Progress/Closed",
 "assignedTo": "userId"
}
```

---

4. Setup Instructions

Prerequisites

- Node.js
- MongoDB
- Git

Installation Steps

```
# Clone repository

git clone https://github.com/your-repo/project.git


# Navigate to project

cd project
```

```
# Install frontend dependencies
cd client
npm install

# Install backend dependencies
cd ../server
npm install
```

---

5. Folder Structure

Client (React Frontend)

```
client/
├── src/
│   ├── components/
│   ├── pages/
│   ├── services/
│   └── App.js
└── package.json
```

Server (Node Backend)

```
server/
```

```
├── models/
├── routes/
├── controllers/
├── middleware/
└── server.js
```

---

6. Running the Application

Frontend

```
cd client
npm start
```

Backend

```
cd server
npm start
```

---

7. API Documentation

Register User

- POST `/api/register`

Request Body:

```
{
 "email": "user@example.com",
```

```
 "password": "123456"
}
```

Login User

- **POST** `/api/login`

Create Ticket

- **POST** `/api/tickets`

Get Tickets

- **GET** `/api/tickets`

Update Ticket

- **PUT** `/api/tickets/:id`

---

8. Authentication

- Uses JWT (JSON Web Token) authentication
- Token generated during login
- Stored in browser local storage
- Middleware verifies token for protected routes
- Role-based authorization ensures proper access

---

9. User Interface

Includes:

- **Login Page**
- **Registration Page**
- **Dashboard**
- **Ticket Creation Form**
- **Ticket Status Tracking**

(Attach screenshots or GIFs here in your report)

---

10. Testing

- **Manual testing for UI and functionality**
- **API testing using Postman**
- **Tested modules:**
  - **Authentication**
  - **Ticket creation & assignment**
  - **Role-based access**

---

11. Screenshots / Demo

- **Add screenshots of:**
  - **Login Page**
  - **Dashboard**
  - **Ticket Creation**
  - **Ticket Assignment**
- **Demo Link: (Add your project demo link)**

---

12. Known Issues

- Limited automation rules
- No AI-based classification
- UI can be further enhanced

---

13. Future Enhancements

- AI-based ticket classification
- Chatbot integration
- Advanced analytics dashboard
- Mobile application support

Conclusion

This MERN stack project successfully implements a full-stack solution for automating ticket assignment, improving efficiency, and enhancing the overall support system experience.